

REVISÃO - AULA 07

EXPRESSÕES, ATRIBUIÇÃO E CONTROLE

Paradigmas de Linguagens de Programação

Baseado nos capítulos 7 e 8 de Sebesta indicados no material da aula

Esta revisão reúne os pontos principais da aula e mantém exemplos dos próprios slides. A ideia é explicar o conteúdo de forma direta, destacando as diferenças entre linguagens e as armadilhas que podem aparecer no código.

1. O que a aula quer mostrar

- Como uma linguagem decide a ordem em que uma expressão é avaliada.
- Como precedência e associatividade alteram o resultado.
- Como conversões automáticas podem esconder erros.
- Como booleanos, atribuições, seleções e laços mudam de uma linguagem para outra.
- Por que alguns erros de fluxo parecem pequenos, mas podem causar problemas grandes.

2. Três estruturas básicas de controle

Estruturas de controle: quantas são necessárias?

Sequência	Seleção	Iteração
uma sentença após a outra	escolher um caminho	repetir enquanto uma condição vale
<pre>a = 1; b = a + 2;</pre>	<pre>if (x > 0) ... else ...</pre>	<pre>while (x > 0) x--;</pre>
Böhm e Jacopini (1966): sequência, seleção e laço lógico bastam para qualquer algoritmo. O goto não é necessário.		

Por que então tantas outras (for, switch, do, foreach)? **Facilidade de escrita.** O limite é a **legibilidade**: quanto maior a linguagem, menos do código alheio você reconhece (p. 329).

Sebesta, 11ª ed. - 98.1 - p. 328-330

Anotação: A aula resume o fluxo em sequência, seleção e iteração. Outras estruturas, como for e switch, existem principalmente para facilitar a escrita e a leitura.

Como estudar este conteúdo

O mais importante não é decorar toda a sintaxe de cada linguagem. O objetivo é olhar um trecho de código e conseguir perceber onde existe risco de interpretação diferente: ordem dos operadores, conversão de tipos, condição booleana, atribuição dentro de if, switch sem break ou laço que não progride.

3. Expressões aritméticas: o que muda entre linguagens

Uma expressão aparentemente simples pode produzir resultados diferentes dependendo das regras da linguagem. Antes de calcular, é preciso saber qual operador tem prioridade e qual é a associatividade.

Expressões aritméticas: questões de projeto

- Quais são as regras de **precedência**?
- Quais são as regras de **associatividade**?
- Em que **ordem os operandos** são avaliados?
- Há restrições a **efeitos colaterais**?
- O usuário pode **sobrecarregar** operadores?
- Que **mistura de tipos** é permitida?

Lisp dispensa a precedência (p. 306)

(+ a (* b c)): notação prefixa, e os parênteses dizem a ordem.

$$a + b * c \quad \text{com} \quad a=3, b=4, c=5$$

da esquerda para a direita

$$(3 + 4) * 5 = 35$$

da direita para a esquerda

$$3 + (4 * 5) = 23$$

com precedência (* antes de +)

$$3 + 4 * 5 = 23$$

APL: sempre da direita para a esquerda (p. 304)

$$3 \times 4 + 5 = 27$$

Sebesta, 11ª ed. - §7.2 e §7.2.1 - p. 300-305

Precedência

Precedência é a prioridade dos operadores. Em uma expressão como $3 + 4 * 5$, a multiplicação normalmente é feita antes da soma, resultando em 23.

Associatividade

Quando os operadores têm o mesmo nível de precedência, a associatividade define a direção. Alguns operadores são avaliados da esquerda para a direita; a potência, em linguagens como Python e JavaScript, é associativa à direita.

- $3 + 4 * 5 = 23$ quando $*$ tem prioridade sobre $+$.
- $(3 + 4) * 5 = 35$ quando os parênteses mudam a ordem.
- Em Lisp, a notação prefixa e os parênteses já deixam a ordem explícita.

Anotação: Antes de tentar resolver uma expressão, olhar primeiro para parênteses, precedência e associatividade.

4. Precedência e associatividade na prática

Precedência e associatividade na prática

Expressão	Python	JavaScript	Java
-2 ** 2	-4	erro de sintaxe	(não há **)
2 ** 3 ** 2	512	512	–
1 + 2 + "3"	TypeError	"33"	"33"
-7 / 2 (inteiros)	-4 (com //)	-3.5	-3
-7 % 2	1	-1	-1

Associatividade (p. 303)

Esquerda: * / % + -
Direita: ** (Python, JS, Ruby, Fortran),
++, --, menos unário
Visual Basic: ^ à esquerda!

Excel

=-2^2 dá 4: lá o menos unário vence o
^.

Na dúvida, **use parênteses** (p. 305): eles deixam a intenção explícita para o compilador e para o próximo leitor.

Sebesta, 11ª ed. · §7.2.1.1-7.2.1.3 · p. 301-305 · exemplos/01_precedencia

O quadro do slide é importante porque mostra que a mesma expressão nem sempre funciona igual em Python, JavaScript e Java.

Exemplos que valem lembrar

- -2 ** 2 em Python resulta em -4. O operador de potência é aplicado antes do sinal.
- 2 ** 3 ** 2 resulta em 512 em Python e JavaScript, pois a potência associa à direita.
- 1 + 2 + "3" mistura números e texto. Em JavaScript e Java, o resultado mostrado é "33"; em Python ocorre erro de tipo.
- -7 / 2 também mostra diferenças: divisão inteira e divisão real não são tratadas da mesma forma em todas as linguagens.
- O operador % com valor negativo também pode variar.

Anotação: A regra prática do próprio material é usar parênteses quando houver dúvida. Além de ajudar o compilador, facilita para quem for ler o código depois.

5. Ordem dos operandos e efeitos colaterais

Efeito colateral é quando uma operação faz mais do que apenas calcular um valor, por exemplo alterando uma variável. Se a ordem dos operandos não for bem definida, o resultado pode depender da linguagem.

Ordem dos operandos e efeitos colaterais

Exemplo do livro (p. 307)

```
int a = 5;

int fun1(void) {
    a = 17;          /* efeito colateral */
    return 3;
}

int main(void) {
    a = a + fun1();  /* 8 ou 20? */
    printf("a = %d\n", a);
}
```

8

Java

a ordem é garantida: da esquerda para a direita (p. 308)

20

C (gcc 14)

a ordem não é definida pelo padrão

20

Go

a especificação não define a leitura de a em relação à chamada

Transparência referencial (p. 308): sem efeitos colaterais, a ordem deixa de importar, e $f(x) + f(x)$ pode virar $2 * f(x)$.

Sebesta, 11ª ed. - §7.2.2 - p. 307-309 - exemplos/02_efeitos_colaterais

Exemplo do slide

A variável a começa com 5. A função fun1 altera a para 17 e retorna 3. A expressão $a = a + \text{fun1}()$ pode depender de quando a leitura de a acontece em relação à chamada da função.

- Java garante a avaliação da esquerda para a direita.
- No exemplo apresentado, C não define essa ordem pelo padrão.
- O slide também cita Go como linguagem em que a especificação não define a leitura de a em relação à chamada.

Transparência referencial

Quando não existem efeitos colaterais, a ordem de avaliação deixa de ser tão problemática. Uma função que sempre devolve o mesmo resultado para a mesma entrada é mais fácil de raciocinar.

Anotação: Evitar escrever uma expressão que lê uma variável e, ao mesmo tempo, chama algo que altera essa mesma variável.

6. Operadores sobrecarregados

Sobrecarga significa que o mesmo operador pode ter significados diferentes dependendo dos tipos envolvidos.

Operadores sobrecarregados

- O mesmo símbolo com **vários significados**: + soma inteiros, reais e concatena (Java)
- Risco (p. 309): & em C++ é E bit a bit e "endereço de"
- **Definida pelo usuário** (p. 404): C++, C#, Python, Rust, Kotlin. Java **não** permite

Java

```
System.out.println('a' + 1);    // 98
System.out.println("" + 'a' + 1); // a1
x.add(y).multiply(tres);    // BigDecimal: sem +
```

Python - sobrecarga com métodos especiais

```
class Vetor:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, outro):
        # v1 + v2
        return Vetor(self.x + outro.x,
                      self.y + outro.y)

    def __mul__(self, k):
        # v * 3
        return Vetor(self.x * k, self.y * k)

v = Vetor(1, 2) + Vetor(3, 4)    # Vetor(4, 6)
print(v * 3)                    # Vetor(12, 18)
```

Sebesta, 11ª ed. · §7.3 · p. 309-310 e §9.11 · p. 404 · exemplos/03_sobrecarga

- O operador + pode somar números e também concatenar texto.
- Em Java, 'a' + 1 produz 98 porque o caractere participa de uma operação numérica.
- Ao começar com uma string vazia, "" + 'a' + 1 produz "a1".
- C++, C#, Python, Rust e Kotlin permitem sobrecarga definida pelo usuário; o material destaca que Java não permite.

Exemplo em Python

O slide mostra uma classe Vetor com os métodos especiais `__add__` e `__mul__`. Assim, o programador define o que significa usar + ou * com objetos dessa classe.

Anotação: Sobrecarga deixa o código mais natural, mas também exige atenção: o símbolo sozinho não diz tudo; é preciso saber quais tipos estão participando da expressão.

7. Conversões de tipos: alargamento, estreitamento e coerção

Conversões de tipos: alargamento, estreitamento e coerção

Java · coerção e cast

```
int grande = 123456789;
float f = grande;      // alargamento implícito
System.out.println((int) f); // 123456792 !

double d = 3.99;
System.out.println((int) d); // 3 (cast)
System.out.println((byte) 200); // -56

byte b = 10, c = 20;
// byte s = b + c; // erro: b + c é int (p. 312)

System.out.println(1 / 2); // 0
System.out.println(1 / 2.0); // 0.5
```

Linguagem	int → real (implícito)	int32 + int64
C	sim	coerção
Java	sim (alargamento)	coerção para long
Python	sim	(um só tipo int)
Go	não: float64(i)	erro de compilação
Rust	não: i as f64	erro de compilação
ML, F# (p. 312)	não	erro de compilação

Coerção esconde erros (p. 312): em `d = b * a`; com `a` digitado por engano no lugar de `c`, o Java converte e compila.

Sebesta, 11ª ed. - §7.4 - p. 310-313 - exemplos/04_conversoes

Alargamento

É a conversão para um tipo que normalmente comporta uma faixa maior de valores. O exemplo `int` para `float` em Java é implícito, mas o slide mostra que ainda pode ocorrer perda de precisão.

Estreitamento e cast

Ao transformar um valor mais amplo em um tipo menor, pode haver perda de informação. Por isso, Java exige `cast` em casos como `int i = (int) 3.7`.

- `(int) 3.99` resulta em 3.
- `(byte) 200` resulta em -56 no exemplo apresentado.
- `byte + byte` produz `int` em Java.
- `1 / 2` resulta em 0; `1 / 2.0` resulta em 0.5.

Anotação: Conversão automática não significa que o valor ficará exatamente igual. É preciso observar precisão, faixa do tipo e divisão inteira.

8. Coerção implícita em JavaScript e PHP

JavaScript e PHP aparecem na aula como exemplos importantes de linguagens que fazem conversões automáticas em várias situações.

Coerção implícita: JavaScript e PHP

JavaScript

```
"5" - 2           // 3
"5" + 2           // "52"
"5" * "2"         // 10
true + 1          // 2
[] + {}           // "[object Object]"
null + 1          // 1
undefined + 1     // NaN
"b" + "a" + "a" + "a" // "baNaNa"

0 == ""          // true
"" == "0"        // false
0 == "0"         // true  (não é transitivo!)
0 === "0"        // false (=== não converte)
```

PHP - "type juggling"

```
var_dump("0e123" == "0e456"); // true !
var_dump("0e123" === "0e456"); // false
var_dump("1" == "01");        // true

md5("240610708") // "0e462097431906..."
md5("QNKCDZO")   // "0e830400451993..."
// == entre os dois hashes: true
```

Regra prática

Em JavaScript e PHP, compare com `===` e `!==`, que **não** fazem coerção.

Sebesta, 11ª ed. - §7.4.1 Coerção em expressões - p. 311-312 - exemplos/04_conversoes

JavaScript

- `"5" - 2` resulta em 3.
- `"5" + 2` resulta em "52".
- `"5" * "2"` resulta em 10.
- `0 == ""` é verdadeiro, enquanto `"" == "0"` é falso.
- `0 == "0"` é verdadeiro.

Comparação estrita

O slide recomenda `===` e `!==` em JavaScript e PHP porque esses operadores não fazem coerção.

PHP e type juggling

O material mostra o caso de strings iniciadas por `0e` que, usando `==`, podem ser tratadas como valores numéricos equivalentes. Com `===`, a comparação considera também o tipo e o conteúdo sem essa coerção.

Anotação: Quando a linguagem converte automaticamente, um código curto pode esconder uma regra importante. Comparação estrita deixa a intenção mais clara.

9. Expressões relacionais e booleanas

Expressões relacionais e booleanas

C · booleano é inteiro (p. 315)

```
int a = 3, b = 2, c = 1;
printf("%d\n", a > b > c); // 0 !
// (3 > 2) vale 1, e 1 > 1 é falso

printf("%d\n", (a > b) && (b > c)); // 1

if (-1) printf("verdadeiro\n"); // != 0
```

Python · comparações encadeadas

```
a, b, c = 3, 2, 1
print(a > b > c) # True: (a > b) and (b > c)

# contam como FALSO em um if:
# 0 0.0 "" [] {} None
# contam como VERDADEIRO:
# "0" [0] " "
```

Linguagem	O que o if aceita
C, Python, JavaScript	qualquer valor (0, vazio e nulo contam como falso)
Java, C#, Go, Rust	somente booleano: if (n) com n inteiro nem compila

Sebesta, 11ª ed. - \$7.5 - p. 313-315 · exemplos/05_booleanas

C
No exemplo do slide, `3 > 2 > 1` resulta em 0. Primeiro `3 > 2` vale 1; depois é feita a comparação `1 > 1`, que é falsa.

- Python**
Python permite comparações encadeadas. Assim, `3 > 2 > 1` equivale a verificar `(3 > 2) and (2 > 1)`, resultando em verdadeiro.
- Em Python, 0, 0.0, string vazia, lista vazia, dicionário vazio e None contam como falso em um if.
 - Valores como "0", [0] e uma string contendo espaço contam como verdadeiro.
 - Java, C#, Go e Rust exigem booleano no if.

10. Avaliação em curto-circuito

Avaliação em curto-circuito

Java · && faz curto-circuito, & não

```
// busca do livro (p. 316)
while (i < lista.length && lista[i] != chave)
    i++; // && evita lista[lista.length]

String s = null;
s != null && s.length() > 0 // false
s != null & s.length() > 0 // NullPointerException
```

JavaScript · ?. e ??

```
const u = { nome: "Ana", endereco: null };
u.endereco?.cidade // undefined, sem erro

const qtd = 0;
qtd || 10 // 10 (o 0 se perdeu!)
qtd ?? 10 // 0 (só troca null/undefined)
```

Cuidado com efeitos colaterais (p. 316)
Em `(a > b) || (b++ / 3)`, o `b++` só acontece quando `a <= b`. Em Python, `0 or "padrão"` devolve "padrão": and e or devolvem um dos **operandos**.

Sebesta, 11ª ed. - \$7.6 - p. 315-317 · exemplos/05_booleanas

Anotação: Em `s != null && s.length() > 0`, a segunda parte só é executada se `s` não for null. Isso evita o acesso inválido mostrado no exemplo.

11. Sentenças de atribuição

Sentenças de atribuição

C · a atribuição é uma expressão (p. 319-320)

```
sum = count = 0;           // múltiplos alvos
a = b + (c = d / b) - 1;    // exemplo do livro

while ((ch = getchar()) != EOF) { ... }

if (x = y) { ... }          // compila! atribui e testa y
```

Python · atribuição múltipla (p. 320)

```
a, b = b, a                # troca sem temporária
if (n := len(dados)) > 5: ... # morsa (Python 3.8+)
```

Linguagem	Atribuição é expressão?
C, C++, JavaScript	sim: if (x = y) compila
Java, C#	sim, mas o if exige booleano
Python	não, exceto o operador morsa :=
Go	não: nem x++ é expressão
Rust	vale (): if x = 5 não compila

Linguagens funcionais (p. 321)

val (ML) e let (F#, Rust) dão **nome** a um valor, que não muda.

Sebesta, 11ª ed. - §7.7 - p. 317-321 - exemplos/06_atribuicao

Em algumas linguagens, atribuição também é uma expressão. Isso permite construções úteis, mas também cria armadilhas.

- Em C, C++ e JavaScript, if (x = y) pode compilar: o código atribui y a x e depois testa o valor.
- Java e C# também tratam atribuição como expressão, mas o if exige booleano.
- Python não usa atribuição comum como expressão, mas possui o operador morsa :=.
- Go não trata atribuição como expressão.

12. Atribuição de modo misto

Atribuição de modo misto

Java · só alargamento (p. 322)

```
double d = 3;              // ok: alargamento
// int i = 3.7;            // erro: estreitamento
int i = (int) 3.7;          // 3 (cast explícito)
long l = i;                 // ok

i += 3.7;                   // COMPILA: cast escondido
System.out.println(i);      // 6
```

C · coerção em qualquer direção

```
int i = 3.7;               // 3, em silêncio
char c = 300;              // 44 (só um aviso)
unsigned int u = -1;        // 4294967295
```

Armadilha clássica

double taxa = acertos / total * 100; com inteiros dá **0.0**: a divisão inteira acontece **antes** do alargamento para double.

Sebesta, 11ª ed. - §7.8 - p. 321-322 - exemplos/06_atribuicao

- double d = 3 é aceito em Java por alargamento.
- int i = 3.7 não é aceito sem cast.
- i += 3.7 compila no exemplo porque o operador composto faz uma conversão que pode ficar escondida.
- double taxa = acertos / total * 100 pode resultar em 0.0 quando acertos e total são inteiros, pois a divisão inteira ocorre antes.

Anotação: Quando o resultado deveria ter casas decimais, conferir os tipos antes da divisão.

13. Seleção: if, else, switch e match

Else pendente

Seleção de dois caminhos: o "else pendente"

C · a indentação engana (p. 331)

```
int sum = 0, count = 5, result = -1;
if (sum == 0)
    if (count == 0)
        result = 0;
else
    result = 1; // de qual if é este else?
printf("%d\n", result); // 1
```

Regra de Java, C, C++ e C#

O else pertence ao if **mais próximo** que ainda não tem else. A indentação é ignorada (exceto em Python e F#).

Chaves obrigatórias eliminam o problema: Perl, Go e Rust (p. 332).

Seletores que devolvem valor

```
let y = if x > 0 { x } else { 2 * x }; // Rust: if é uma expressão (como em F#, p. 334)
y = x if x > 0 else 2 * x # Python: expressão condicional (p. 306)
```

Sebesta, 11ª ed. · §8.2.1 · p. 330-334 · exemplos/07_selecao

Em Java, C, C++ e C#, o else pertence ao if mais próximo que ainda não possui else. A indentação não altera essa regra.

Anotação: Usar chaves evita depender apenas da indentação para entender a estrutura.

Seleção múltipla

Seleção múltipla: do switch de C ao match

C · sem break, o controle "cai" (p. 335)

```
switch (index) {
    case 1:
    case 3: odd += 1;
    case 2:
    case 4: even += 1;
    default: printf("Error\n");
}
// index = 1: odd, even E erro!
```

Java 14+ · switch como expressão

```
String tipo = switch (dia) {
    case SAB, DOM -> "fim de semana";
    case SEG, TER, QUA,
    QUI, SEX -> "dia útil";
}; // sem break, sem cair;
// exaustivo para enum
```

Rust · match exaustivo

```
match n {
    i32::MIN..=-1 => "negativo",
    0 => "zero",
    1..=9 => "um dígito",
    _ => "vários dígitos",
}
// sem o _ : não compila
```

Go: cada caso termina sozinho, e cair exige fallthrough. C#: cair implicitamente é proibido (p. 336). Python 3.10: match com padrões e guardas.

Sebesta, 11ª ed. · §8.2.2 · p. 334-341 · exemplos/07_selecao

- No switch tradicional de C, a falta de break faz o controle continuar nos casos seguintes.
- Java 14+ possui switch como expressão usando ->.
- Rust usa match e exige cobertura exaustiva; o exemplo usa _ para tratar os demais valores.
- Go encerra cada caso sozinho; fallthrough precisa ser explícito.

Anotação: A armadilha principal do switch de C é esquecer que um case pode continuar executando os próximos.

14. Laços controlados por contador

Laços controlados por contador

C - o for é um laço lógico disfarçado

```
for (int i = 0; i < 10; i++) {  
    printf("%d ", i);  
    i += 2;           // altera a contagem  
}  
  
for (; i <= 100; ) // expressões opcionais  
    soma += i++;
```

Python - o for percorre um iterável

```
for i in range(0, 10):  
    print(i, end=" ")  
    i += 2           # NÃO muda nada  
                    # 0 1 2 3 4 5 6 7 8 9  
  
for i in range(10, 0, -3): # 10 7 4 1  
    ...
```

Questões de projeto (p. 342)

Qual é o tipo e o escopo da variável? Posso alterá-la no corpo? Os parâmetros são avaliados uma vez ou a cada volta?

Sebesta, 11ª ed. - §8.3.1 - p. 342-345 - exemplos/08_iteracao

O for de C e o for de Python parecem semelhantes, mas funcionam de maneira diferente.

- Em C, alterar `i` dentro do corpo afeta a contagem.
- No exemplo, `i += 2` dentro do for faz a saída ser 0, 3, 6, 9.
- Em Python, alterar `i` dentro do corpo não muda a sequência produzida pelo range na próxima iteração.
- `range(10, 0, -3)` produz 10, 7, 4, 1.

15. Laços lógicos, saídas e iteradores

Laços lógicos, saídas e iteradores

Java - break rotulado (p. 348)

```
externo:  
for (int i = 0; i < m.length; i++)  
    for (int j = 0; j < m[i].length; j++)  
        if (m[i][j] < 0) {  
            System.out.println(i + ", " + j);  
            break externo; // sai dos DOIS laços  
        }
```

JavaScript - for...in × for...of

```
const notas = [10, 20, 30];  
let total = 0;  
for (const i in notas) total += i;  
console.log(total); // "0012": chaves!  
  
total = 0;  
for (const v of notas) total += v;  
console.log(total); // 60: valores
```

Zune (2008) - o laço lógico sem progresso

```
while (dias > 365) {  
    if (bissexto(ano)) {  
        if (dias > 366) { dias -= 366; ano++; }  
        // dias == 366: nada muda, laço infinito!  
    } else { dias -= 365; ano++; }  
}
```

Pós-teste (do-while) executa **ao menos uma vez**, e isso é fácil de esquecer (p. 347).

Sebesta, 11ª ed. - §8.3.2-8.3.4 - p. 345-352 - exemplos/08_iteracao

- Java possui `break` rotulado, que pode sair de dois laços aninhados.
 - Em JavaScript, `for...in` percorre chaves/índices; `for...of` percorre valores.
 - No exemplo, somar os índices com `for...in` produz a string "0012"; com `for...of`, a soma dos valores é 60.
- Anotação:** Antes de usar um for, entender exatamente o que ele percorre: contador, índices, chaves ou valores.

16. Laço sem progresso e o caso Zune

Um laço precisa alterar alguma informação que faça a condição caminhar para o fim. Se existir um caso em que nada muda, o programa pode entrar em repetição infinita.

Laços lógicos, saídas e iteradores

Java · break rotulado (p. 348)

```
externo:
for (int i = 0; i < m.length; i++)
  for (int j = 0; j < m[i].length; j++)
    if (m[i][j] < 0) {
      System.out.println(i + ", " + j);
      break externo; // sai dos DOIS laços
    }
```

JavaScript · for...in × for...of

```
const notas = [10, 20, 30];
let total = 0;
for (const i in notas) total += i;
console.log(total); // "0012": chaves!

total = 0;
for (const v of notas) total += v;
console.log(total); // 60: valores
```

Zune (2008) · o laço lógico sem progresso

```
while (dias > 365) {
  if (bissexto(ano)) {
    if (dias > 366) { dias -= 366; ano++; }
    // dias == 366: nada muda, laço infinito!
  } else { dias -= 365; ano++; }
}
```

Pós-teste (do-while) executa **ao menos uma vez**, e isso é fácil de esquecer (p. 347).

Sebesta, 11ª ed. · §8.3.2-8.3.4 · p. 345-352 · exemplos/08_iteracao

No exemplo do Zune apresentado no slide, quando dias era exatamente 366 em um ano bissexto, a condição interna dias > 366 era falsa. Como dias e ano não mudavam, o while continuava para sempre.

Anotação: Ao revisar um while, procurar o caminho em que a condição continua verdadeira mas nenhuma variável usada nela é modificada.

17. Desvio incondicional: goto

Desvio incondicional: o goto

"goto fail" (Apple, 2014) · versão simplificada

```
int valida_certificado(void) {
  int err;
  if ((err = verifica_hash()) != 0)
    goto fail;
  goto fail; // duplicada: desvia SEMPRE
  if ((err = verifica_assinatura()) != 0)
    goto fail; // nunca é alcançada
fail:
  return err; // 0 = "certificado válido"
}
```

Dijkstra (1968)

"Go To Statement Considered Harmful"

Quem tem goto hoje

C, C++, C#, Go. Java não tem (p. 353)

Gotos restritos

break, continue e return melhoram a legibilidade (p. 353)

Com chaves obrigatórias (Go, Rust) ou com gcc -Wall (aviso de endentação enganosa), o bug teria sido pego.

Sebesta, 11ª ed. · §8.4 · p. 352-353 · exemplos/09_desvio_protegido

O exemplo 'goto fail' mostra uma linha goto fail duplicada. Isso fazia o programa desviar sempre antes da verificação de assinatura.

- C, C++, C# e Go possuem goto.
- Java não possui goto.
- O material apresenta break, continue e return como desvios mais restritos e normalmente mais legíveis.

Anotação: O problema não é apenas existir um desvio; é quando o fluxo fica difícil de acompanhar e uma parte importante do código deixa de ser executada.

18. Comandos protegidos de Dijkstra

Comandos protegidos de Dijkstra

```
if x >= y -> max := x
[] y >= x -> max := y
fi
```

Exemplo do livro (p. 354): se $x = y$, qualquer ramo serve.

- Todas as **guardas** são avaliadas
- Várias verdadeiras: escolha **não determinística**
- Nenhuma verdadeira: erro (no if) ou fim do laço (no do ... od)
- Base do **CSP** de Hoare (1978) e do select de Go

Go · select, herdeiro dos comandos protegidos

```
a <- "a" // os dois canais
b <- "b" // estão prontos

select { // Go sorteia uma guarda
case x := <-a:
    contagem[x]++
case x := <-b:
    contagem[x]++
}

// 1000 repetições:
// map[a:505 b:495] (muda a cada execução)
```

Sebesta, 11ª ed. · §8.5 · p. 353-355 · exemplos/09_desvio_protegido

Os comandos protegidos usam guardas. Todas são avaliadas e, se mais de uma for verdadeira, a escolha pode ser não determinística.

- Se nenhuma guarda for verdadeira em um if protegido, ocorre erro.
- Em um laço protegido, nenhuma guarda verdadeira encerra o laço.
- O material relaciona essa ideia ao CSP de Hoare e ao select de Go.

Anotação: Esse tópico mostra uma forma diferente de pensar seleção: em vez de uma ordem fixa de if/else, existem guardas que determinam quais caminhos estão disponíveis.

19. Exercícios da aula - onde estão as armadilhas

Exercício em duplas: encontre o bug (1/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

1 · JavaScript

```
const precos = [10, 20, 30];
let total = 0;
for (const p in precos)
    total += p;
console.log("Total: " + total);
```

2 · Python

```
def desconto(preco, d=None):
    d = d or 10 # padrão: 10%
    return preco * (100 - d) / 100

print(desconto(200, 0))
```

Arquivos: exemplos/10_exercicio (q1.js ... Q6.java) · execute em onecompiler.com

- JavaScript: for...in usa índices, não os preços.
- Python: $d = d \text{ or } 10$ troca também o valor 0 pelo padrão 10.

Exercício: encontre o bug (2/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

3 · Java

```
int dia = 2; String nome = "";
switch (dia) {
    case 1: nome = "domingo";
    case 2: nome = "segunda";
    case 3: nome = "terça";
    default: nome = "inválido";
}
System.out.println(nome);
```

4 · C

```
int saldo = 100, saque = 50;
if (saque > saldo);
    printf("saldo insuficiente\n");
printf("fim\n");
```

Arquivos: exemplos/10_exercicio (q1.js ... Q6.java) - execute em onecompiler.com

- Java: switch sem break continua executando os próximos casos.
- C: o ponto e vírgula depois do if encerra a condição; o printf fica fora dela.

20. Exercícios finais e revisão geral

Exercício: encontre o bug (3/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

5 · Go

```
x := 10
if x > 5 {
  x := x * 2
  fmt.Println("dentro:", x)
}
fmt.Println("fora:", x)
```

6 · Java

```
int acertos = 7, total = 10;
double taxa = acertos / total * 100;
System.out.println(taxa + "%");
```

Arquivos: exemplos/10_exercicio (q1.js ... Q6.java) · execute em onecompiler.com

Questão em Go

Dentro do if aparece `x := x * 2`. O `:=` cria uma nova variável `x` naquele bloco. Por isso, o valor impresso dentro é diferente do `x` usado fora.

Questão em Java

`double taxa = acertos / total * 100` usa `acertos` e `total` inteiros. A divisão acontece primeiro como inteira. Com `7 / 10`, o resultado intermediário é 0; depois ele é convertido para `double`.

Anotação: Esses exercícios juntam várias ideias da aula: escopo, atribuição, divisão inteira, switch, laços e valores padrão.

21. Síntese da aula

Síntese: armadilhas e como as linguagens ajudam

Tema	Armadilha	Como as linguagens ajudam
Precedência	<code>-2 ** 2, 1 + 2 + "3"</code>	parênteses; JS obriga <code>(-2) ** 2</code>
Operandos	<code>a + f()</code> com efeito colateral (C, Go)	Java fixa a ordem; funções puras
Conversões	coerção silenciosa (C, JS, PHP)	Go e Rust exigem conversão explícita; <code>===</code>
Booleanos	<code>0, "", []</code> são falsos; <code>a > b > c</code> em C	<code>if</code> só com booleano (Java, Go, Rust); <code>??</code>
Atribuição	<code>if (x = y)</code>	atribuição não é expressão (Python, Go, Rust)
Seleção	<code>switch</code> sem <code>break</code> ; <code>else</code> pendente	<code>-></code> , <code>match</code> exaustivo, chaves obrigatórias
Laços	<code>for...in</code> no JS; laço sem progresso	<code>for...of</code> , iteradores, <code>break</code> rotulado
Desvio	<code>goto</code> sem restrição	<code>break</code> , <code>continue</code> , <code>return</code> : <code>gotos</code> restritos

Sebesta, 11ª ed. · Caps. 7 e 8 · p. 299-361

O último quadro reúne as principais armadilhas: precedência, efeitos colaterais, coerção, booleanos, atribuição, seleção, laços e `goto`.

Checklist rápido antes da prova

- Se houver vários operadores: verificar precedência, associatividade e parênteses.
- Se houver tipos diferentes: verificar se existe coerção ou cast.
- Se houver if: verificar o que a linguagem considera verdadeiro e falso.
- Se houver && ou ||: lembrar do curto-circuito.
- Se houver = dentro de condição: conferir se é atribuição ou comparação.
- Se houver switch: conferir break ou a regra equivalente da linguagem.
- Se houver laço: confirmar o que está sendo percorrido e se existe condição real de parada.
- Se houver função dentro de expressão: verificar se ela altera algum estado usado na própria expressão.